TEST REPORT

Brute-Force Password Cracker

Multi-threaded password recovery · WPF / .NET 8 / C#

Author: [Your full name]

Student ID: [Your ID]

Course / Group: [Course — Group]

Date: 4 June 2026

GitHub repository: [https://github.com/<you>/BruteForceApp]

Each task committed as a separate version (v1.0 – v1.x) — see commit history & README.md

1. Project Overview

This application recovers a hashed password by brute force. A random password is created and hashed with **SHA-256 + a constant static salt**; the attacker side then searches every possible character combination — starting from length 1 — across multiple threads until the matching plaintext is found. The program is built with **WPF on .NET 8** and has a graphical interface for password creation, starting/stopping the attack, live progress, elapsed time, and the recovered password. It also benchmarks **single-threaded vs multi-threaded** execution.

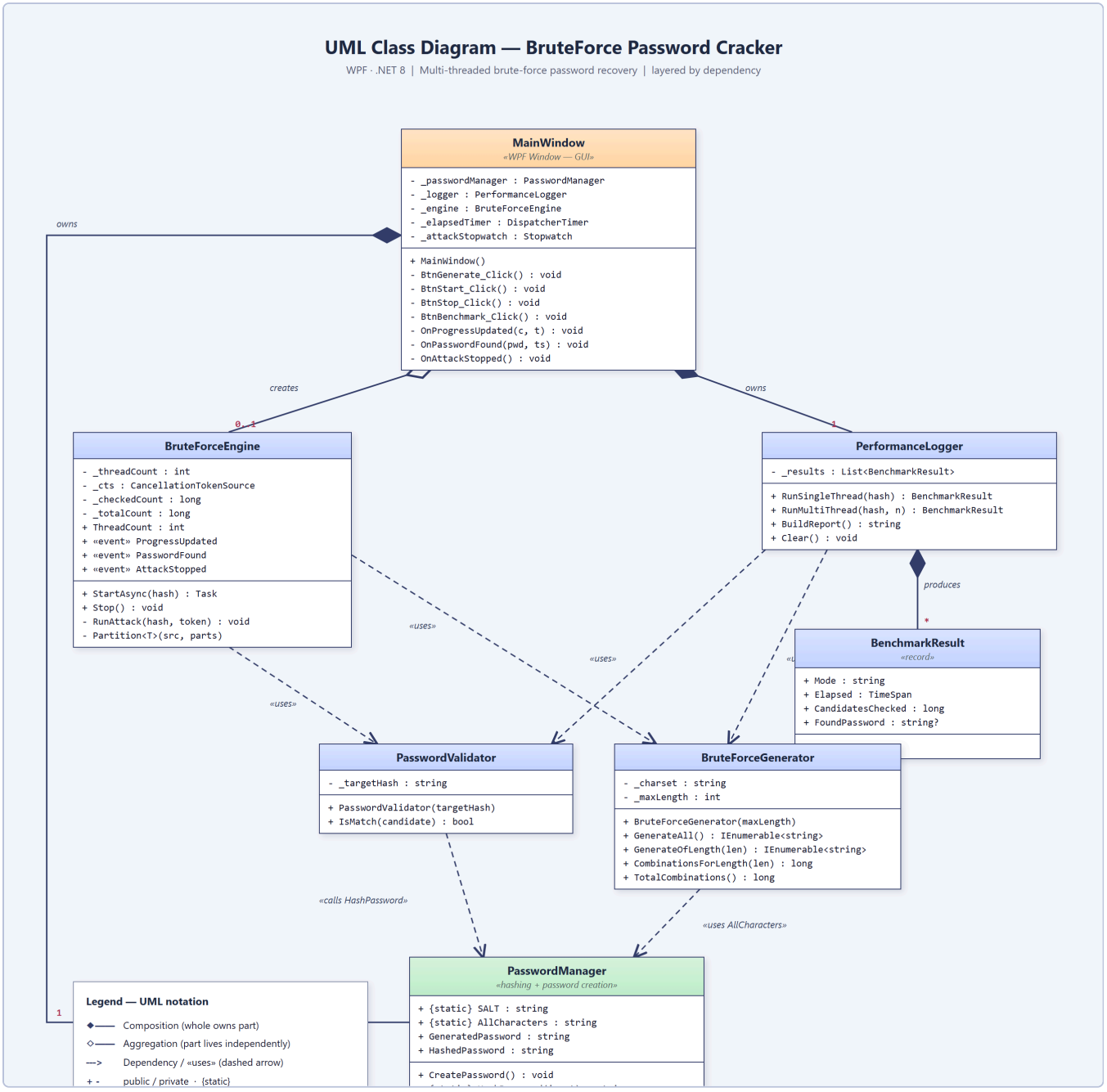
Following the requirements, **each major responsibility lives in its own class and file**, and the brute-force **generator** and **validator** are implemented independently.

Class / file structure

File	Class	Responsibility
`PasswordManager.cs`	`PasswordManager`	Creates the random password (length [4,6)) and hashes any string with SHA-256 + static salt.
`PasswordValidator.cs`	`PasswordValidator`	Independently checks whether a candidate's hash equals the target hash.
`BruteForceGenerator.cs`	`BruteForceGenerator`	Generates all combinations from length 1 → 6; maps an index to a candidate for partitioning.
`BruteForceEngine.cs`	`BruteForceEngine`	Coordinates the multi-threaded attack: partitions the search space, raises progress/found events, stops all threads on success.
`PerformanceLogger.cs`	`PerformanceLogger`	Runs and logs the single-thread vs multi-thread benchmark and computes the speedup.
`MainWindow.xaml(.cs)`	`MainWindow`	The GUI; wires the classes together and updates the display on the UI thread.

2. UML Class Diagram

The diagram below shows the classes, their attributes and methods, and the relationships between them — composition (filled diamond), aggregation (hollow diamond) and dependency (dashed «uses» arrow), with multiplicities.



UML class diagram — layered by dependency. MainWindow (GUI) owns the manager and logger and creates the engine; engine and logger depend on the independent validator and generator; both rely on PasswordManager for hashing and the alphabet.

3. Functionality — Requirement by Requirement

Every item from the brief is implemented. The table maps each requirement to where and how it is met.

Requirement	Status	How it is implemented
UML class diagram	**Done**	Section 2; sources `UML_ClassDiagram.puml/.html/.png`.
GUI: password creation, start/stop, progress, elapsed time, result	**Done**	`MainWindow.xaml` — Generate button, Start/Stop, progress bar + %, elapsed timer, found-password field, log pane.
Each major function in a separate class/file	**Done**	Six classes in six files (see Section 1).
(a) SHA-256 with a constant static salt	**Done**	`PasswordManager.HashPassword()` hashes `SALT + input`; `SALT` is a `const`.
(b) Password length random in [4,6)	**Done**	`_random.Next(4, 6)` → 4 or 5 characters.
(c) Brute force from length 1 up to 6, length unknown in advance	**Done**	Engine loops `length = 1 → 6`; it never reads the real length.
(d) Multi-threading (Thread / Task)	**Done**	Engine uses `Task`; the benchmark also uses raw `Thread`.
(e) Use at most (CPU cores − 1)	**Done**	`Math.Max(1, Environment.ProcessorCount - 1)` → 7 threads on this 8-core machine.
(f) GUI: start/stop, progress, elapsed time, found password	**Done**	See the screenshots in Section 5.
5. Demonstrate parallel (not sequential) execution	**Done**	Threads search disjoint index ranges simultaneously; the benchmark shows multi-thread checks *more* candidates in *less* wall-clock time.
6. Stop all threads immediately once found	**Done**	A shared `CancellationTokenSource` is cancelled on the first match; every thread checks the token each iteration.
7. Generator and validator separate & independent	**Done**	`BruteForceGenerator` produces candidates; `PasswordValidator` only compares hashes — neither references the other.
8. Log single-thread vs multi-thread performance	**Done**	`PerformanceLogger` + the "⚡ Benchmark" button; results in Section 4.

Key design points

- **Hashing:** `SHA256.HashData(UTF8(SALT + candidate))`, hex-encoded. The salt is a single constant shared by creation and verification.
- **Independent search space:** the engine treats the alphabet (a–z, 26 chars) as a number base. Candidate *i* of a given length is the base-26 representation of *i* (`IndexToCombination`). This lets each thread own a contiguous slice `[start, end)` of the space and generate candidates on the fly — true parallelism with no shared list.
- **Stop-on-found:** the matching thread cancels the shared token; all other threads observe it and return within one iteration.

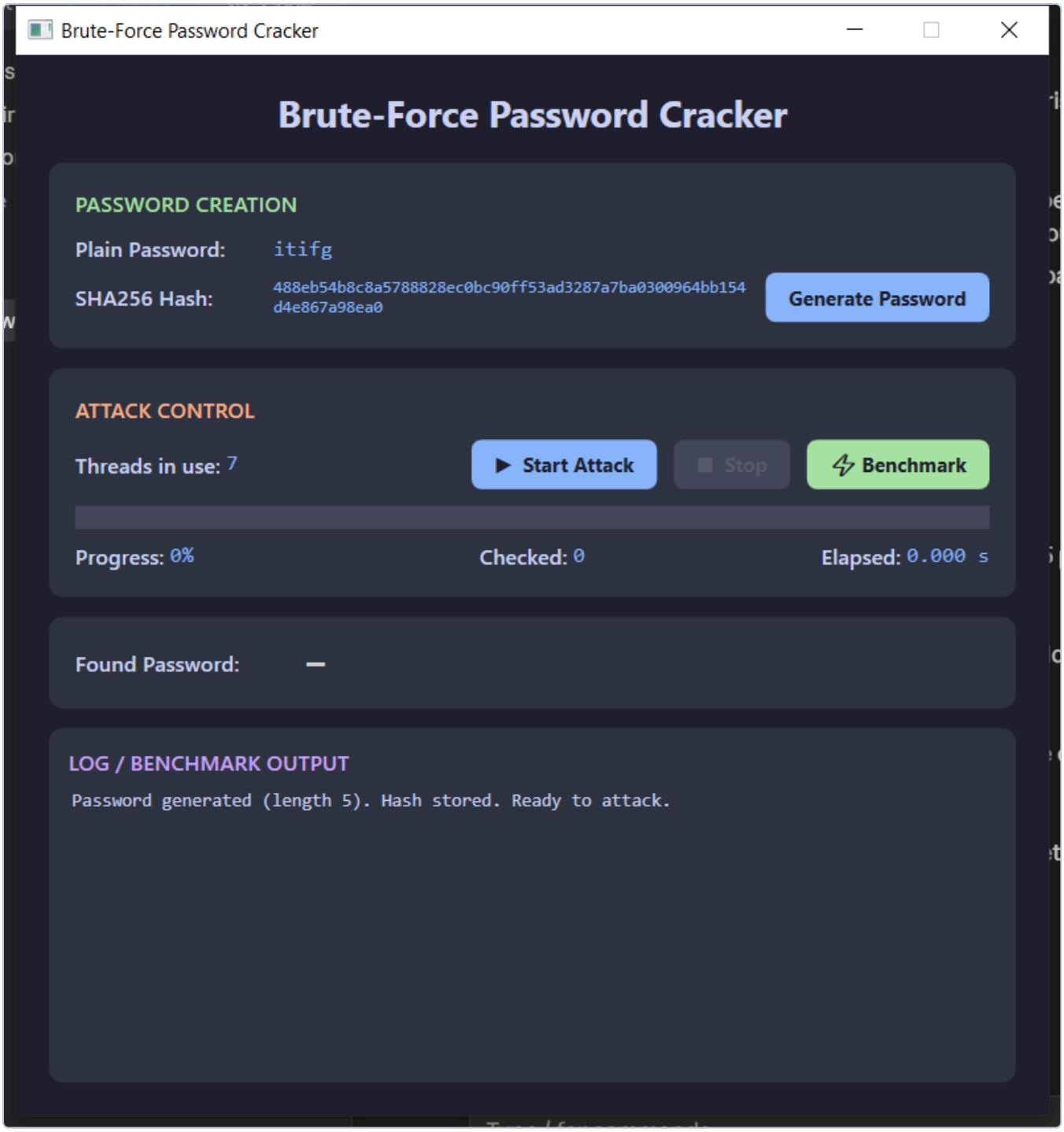
4. Performance: Single-thread vs Multi-thread

The benchmark runs the same brute force twice against the same hash — first on one thread, then on `CPU cores - 1` threads — and records the wall-clock time and the number of candidates checked. Example run (machine: 8 logical cores → 7 worker threads; recovered password `itifg`, length 5):

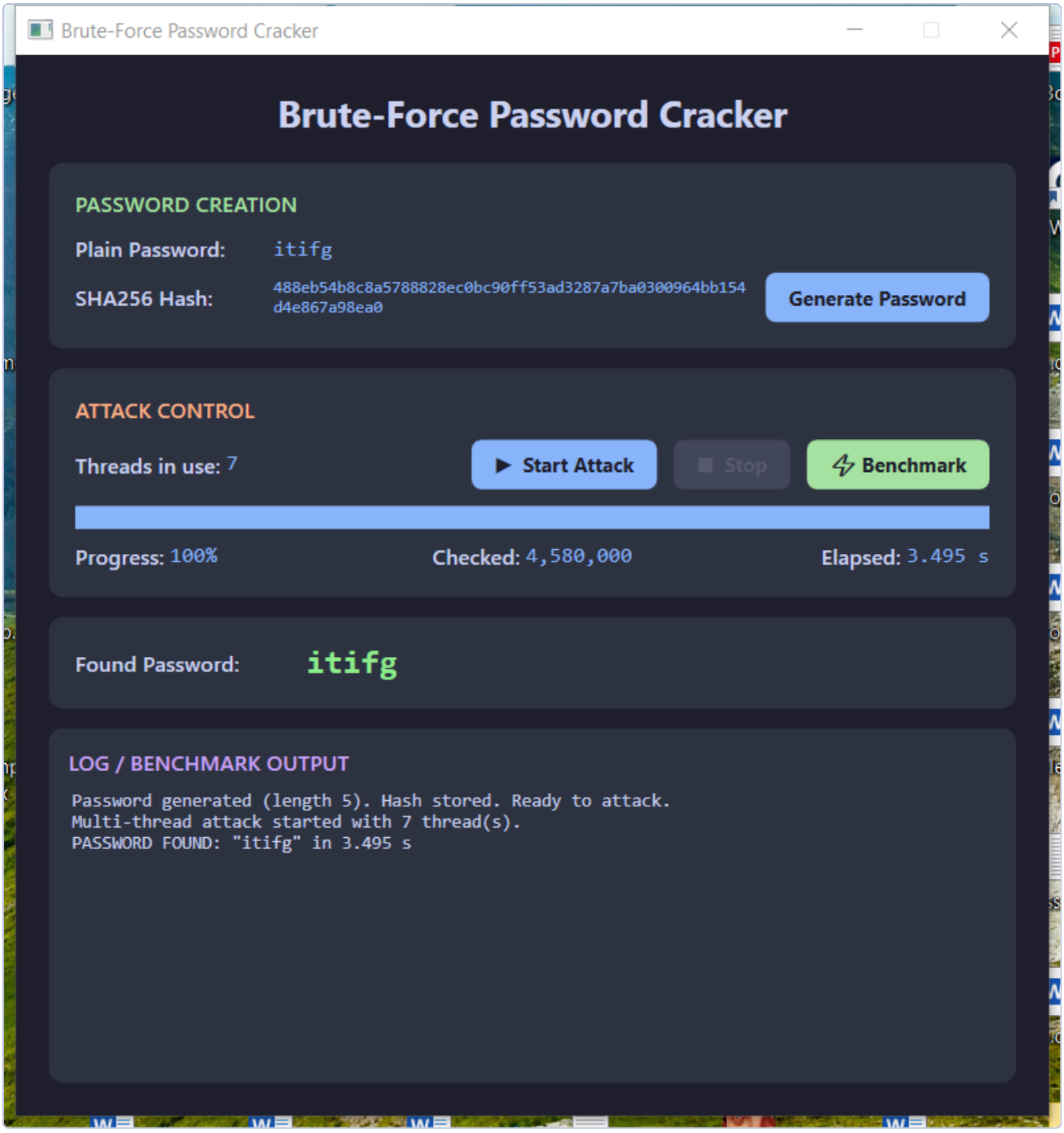
Mode	Threads	Time elapsed	Candidates checked	Result
Single-thread	1	4.446 s	4,470,551	`itifg`
Multi-thread	7	**1.783 s**	4,714,812	`itifg`
Speedup (single ÷ multi)		**2.49×**		

The multi-threaded run checked *more* candidates (4.71M vs 4.47M) yet finished in well under half the time. That is the signature of genuine parallel execution: several threads scan different parts of the keyspace at once, so more total work is done per second. The speedup is below the theoretical 7× because the workload is small (seconds), so thread start-up and the early short lengths add fixed overhead; the gap widens for larger search spaces.

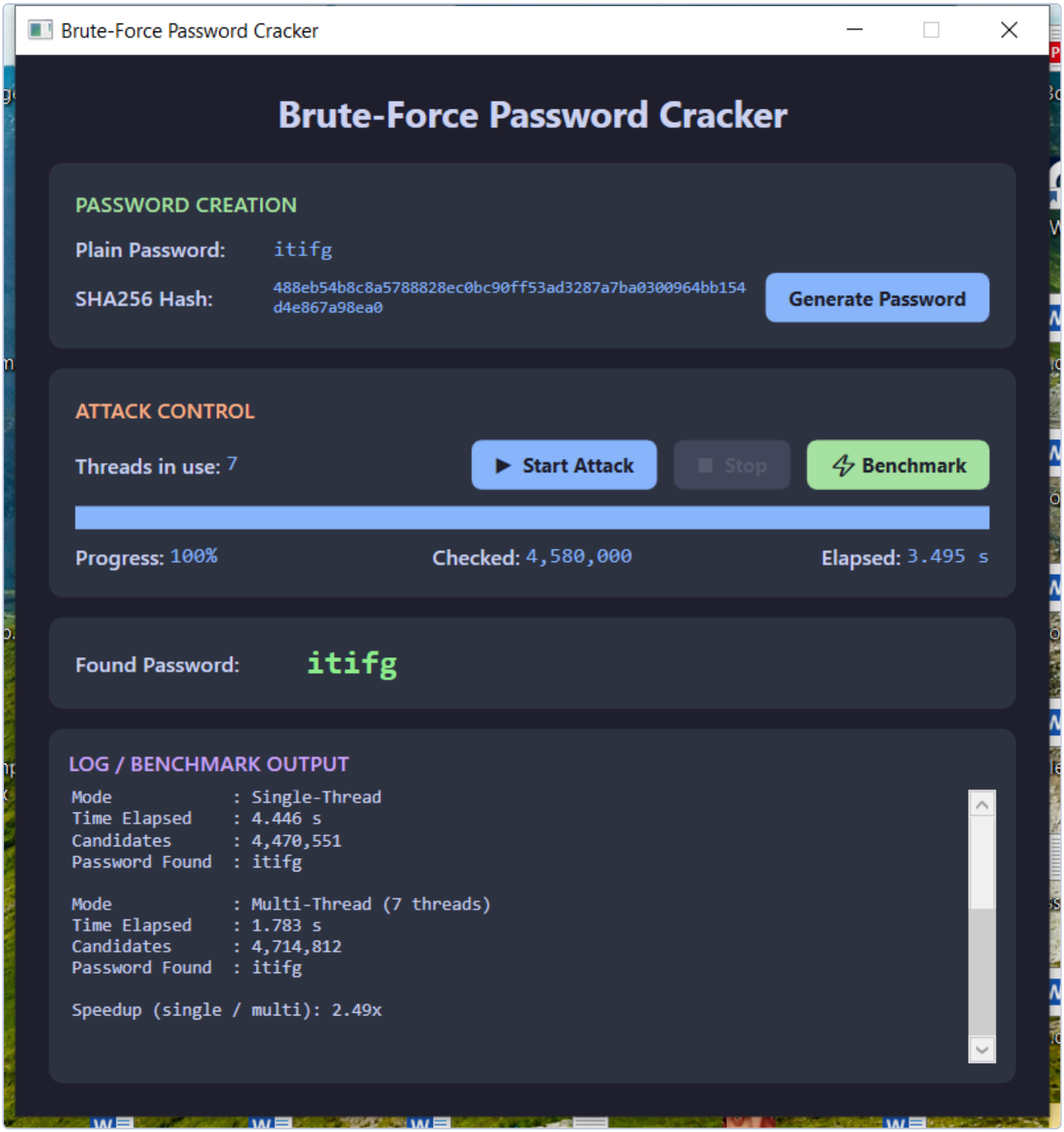
5. Screenshots of the Running Program



(1) After "Generate Password": a random plaintext and its SHA-256 hash are shown; the app reports 7 threads will be used.



(2) After "Start Attack": the password `itifg` is recovered in 3.495 s after checking 4,580,000 candidates; progress reaches 100%.



(3) After "⚡ Benchmark": the log shows the single-thread vs multi-thread comparison and the 2.49× speedup.

6. Challenges Faced

- **Crash the instant the password was found.** The engine waited on its worker tasks with `Task.WaitAll(tasks, token)`. When a thread found the password it cancelled that same token, so `WaitAll` threw `OperationCanceledException`, which propagated through the awaited call onto the UI thread and closed the window. *Fix:* wait with a plain `Task.WaitAll(tasks)` (threads already exit on cancellation) and guard the awaited call.
- **Memory blow-up from materialising candidates.** The first version built a `List/Queue` of every candidate of a length — millions of strings (hundreds of MB) for length 5+. *Fix:* index-range partitioning with `IndexToCombination`, so each thread generates its slice on the fly with no large allocation.
- **Meaningless progress bar.** Measuring progress against the full length-1→6 keyspace (~321M) left a length-4/5 password showing ~0%. *Fix:* the denominator grows to include each length as it is reached, and is pinned to 100% on success.

- **Demonstrability vs. key space.** With the full 62-char alphabet, a length-5 password is ~916M combinations — too slow to demo or benchmark live. The brief fixes the *length* but not the *alphabet*, so a 26-char lowercase set was chosen, keeping a length 4–5 crack to a few seconds.
- **Capturing real screenshots.** The states above were produced by driving the live WPF app through Windows UI Automation (`capture-screens.ps1`) rather than staged mock-ups.

7. How to Build & Run

```
cd BruteForceApp
dotnet run            # or open BruteForceApp.csproj in Visual Studio and press F5

# Compiled debug build:
bin\Debug\net8.0-windows\BruteForceApp.exe
```

Repository version history: `v1.0 implementation` `v1.1 UML diagram` `v1.2 fixes + benchmark/screenshots` — see `README.md` and the Git commit log.